gc_content.py 사용법

FASTQ 파일의 각 read마다 GC 비율과 평균 품질을 함께 계산한다.
그다음 GC 비율이 극단적으로 높은 read N개(기본 10개)와 낮은 read N개를 골라,
각 묶음의 평균 품질이 전체 평균과 얼마나 다른지 비교한다.

1. 실행 방법

lab11/ 폴더에서 실행한다.

```
cd lab11
# 요약만 화면에 출력
python3 src/gc_content.py data/SRR1553607_1.fastq
# read별 결과를 파일로 저장 + 요약을 텍스트로 저장
python3 src/gc_content.py data/SRR1553607_1.fastq doc/gc_content.tsv \
    > doc/gc_content_summary.txt
# 저장하지 않고 극단 read를 20개씩 고르기
python3 src/gc_content.py data/SRR1553607_1.fastq - 20
```

인자	설명
<FASTQ파일>	입력 FASTQ (필수, .gz 압축 파일도 가능)
[read별결과.tsv]	read별 결과를 저장할 파일. 생략하거나 - 이면 저장하지 않음
[N]	고를 극단 read 수 (기본 10)

2. 출력 형식

(1) read별 결과 파일 (헤더 1줄 + read마다 1줄, 탭 구분)

```
name    length    gc_percent    mean_quality
SRR1553607.1    101    42.57    37.44
...
```

(2) 화면 출력 (요약)

```
# data/SRR1553607_1.fastq
read 수: 203445
GC 비율 평균: 58.84% (표준편차 10.15)
전체 평균 품질: 33.45 (표준편차 6.36)

## GC 비율이 가장 높은 read 10개
이름    길이    GC(%)    평균품질
SRR1553607.53820    101    98.02    11.10
...
→ 평균 품질 17.29 (전체 대비 -16.16, 전체 표준편차의 -2.54배)

## GC 비율이 가장 낮은 read 10개
...
```

3. 계산 원리

1. FASTQ를 4줄씩 읽는다 (@이름 / 서열 / + / 품질 문자열). 이름은 헤더의 첫 단어이다.
2. GC 비율 = (G 개수 + C 개수) / read 길이 × 100. 소문자도 대문자로 바꿔서 센다. N 염기도 길이에 포함한다.

3. 평균 품질 = 품질 문자마다 ord(문자) - 33 을 구해 평균을 낸다 (Phred+33).
4. 전체 평균 품질은 read별 평균 품질의 평균이고, 표준편차도 함께 구한다.
5. read를 GC 비율 순으로 정렬해 앞 N개(가장 낮음)와 뒤 N개(가장 높음)를 고른다. GC 비율이 같은 read는 파일 순서를 유지한다.
6. 각 묶음의 평균 품질에서 전체 평균을 빼서 차이를 구하고, 그 차이가 전체 표준편차의 몇 배인지도 보여 준다.

4. 오류 처리

상황	메시지	종료 코드
인자 없음 / N이 1 이상의 정수가 아님	사용법 / 오류: N은 1 이상의 정수여야 합니다	1
입력 파일 없음 / 결과 파일을 쓸 수 없음	오류: 파일을 찾을 수 없습니다 등	2
FASTQ 형식 아님 / 서열·품질 길이 다름 / read 없음	오류: ...	3

read 수가 N보다 적으면 있는 read 수만큼만 고른다. 품질이 모두 같아 표준편차가 0이면 "몇 배" 값은 출력하지 않는다.

5. 테스트 결과

(1) 손으로 계산 가능한 작은 파일 (data/test_gc.fastq)

read	서열	품질	GC(%)	평균 품질
r1	GGGG	IIII	100	40
r2	AAAA	!!!!	0	0
r3	ACGT	5555	50	20
r4	gcat	++++	50	10

- 전체 평균 품질 (40+0+20+10)/4 = 17.50, 표준편차 14.79
- N=1: 고GC r1 → +22.50 (+1.52배), 저GC r2 → -17.50 (-1.18배)
- 기대값과 출력이 모두 일치했다. 소문자 서열(r4)도 GC 50%로 계산되었다.
- 오류 상황(인자 없음, 파일 없음, N이 문자, FASTA 입력, 쓸 수 없는 경로)과 저장하지 않는 경우(두 번째 인자 -), 표준편차가 0인 경우도 확인했다.

(2) 실제 데이터 data/SRR1553607_1.fastq

read 203,445개, 실행 시간 약 2초. 전체 평균 품질 33.45 (표준편차 6.36).

묶음	GC 범위	평균 품질	전체 대비
GC 가장 높은 10개	92.08 ~ 98.02%	17.29	-16.16 (-2.54배)
GC 가장 낮은 10개	12.87 ~ 15.84%	38.33	+4.88 (+0.77배)

(3) 트리밍 후 데이터 data/trimmed.fastq

묶음	평균 품질	전체 대비
전체 (188,357개)	34.95	(표준편차 3.83)
GC 가장 높은 10개	32.02	-2.93 (-0.76배)
GC 가장 낮은 10개	38.46	+3.51 (+0.92배)

6. 해석

GC가 극단적으로 높은 read는 평균 품질이 뚜렷하게 낮다. 전체보다 16점 낮고, 차이가 표준편차의 2.5배이다.
GC가 극단적으로 낮은 read는 오히려 전체보다 약간 높다.

- 고GC read 10개 중 8개는 품질 문자열 뒤쪽이 #(Q2)로 채워져 있고, 그중 7개는 15~53번째 염기부터 끝까지 # 이다. 예: SRR1553607.53820 은 GC 98%, 35번째 염기부터 끝까지 #.
GC가 아주 높은(G·C가 연속되는) 서열은 Illumina 시퀀싱에서 신호가 나빠지기 쉽다.
또 이런 read에는 실제 서열이 아닌 잡음(artifact)이 섞여 있을 수 있다.
- read 10개만으로는 우연일 수 있으므로, GC 구간 전체로도 확인했다(read별 결과 파일로 계산).

GC 구간	read 수	평균 품질
10-20%	54	37.61
20-30%	1,559	37.59
30-40%	8,998	36.61
40-50%	27,096	33.53
50-60%	61,447	33.84
60-70%	79,766	33.38
70-80%	23,491	31.41
80-90%	1,007	26.66
90-100%	27	20.01

GC 비율이 70%를 넘으면서 품질이 계속 떨어지므로, 10개 묶음의 결과가 우연이 아님을 알 수 있다.
상위·하위 1%(각 2,034개)로 넓혀도 고GC 27.54, 저GC 37.57로 같은 경향이다.
- fastp로 트리밍한 뒤에는 차이가 -2.93으로 줄었다. 저품질 꼬리(#)가 잘려 나갔기 때문이다.
- 주의(동점): GC 92.08%인 read가 6개라서, 10개를 고를 때 그중 일부만 포함된다.
GC 92.08% 이상 13개를 모두 포함해도 평균 품질은 18.30으로 결론은 같다.

7. 생성 파일

파일	내용
src/gc_content.py	스크립트
data/test_gc.fastq	테스트용 작은 FASTQ
doc/gc_content.tsv, doc/gc_content_summary.txt	SRR1553607_1 read별 결과 / 요약
doc/gc_content_trimmed.tsv, doc/gc_content_trimmed_summary.txt	trimmed read별 결과 / 요약
doc/gc_content.md, .pdf	이 사용법 문서